
SPECIALTY
common curriculum

Summer Internship

TITLE

Post-Quantum Password Vault

*A hybrid classical + post-quantum secure sync architecture built on an
offline-first, zero-network-attack-surface core vault*

Emphasis of this report:

*For every major cryptographic and architectural decision in this project,
this report states the specific problem that motivated it, the alternatives that
were considered and rejected, and the limitations that remain even after
adopting it.*

HOST ORGANIZATION
MAE Assurance Tunisie

REALIZED BY
Rayan Mohamed Younes

PROFESSIONAL SUPERVISOR
Ms. Rihab Aroui

ACADEMIC YEAR 2025/2026

Contents

1	Executive Summary	3
2	Problem Statement	4
2.1	The problem a password manager exists to solve	4
2.2	The problem that motivated the sync module	4
3	System Architecture	5
4	Core Vault: Design and Limits	6
4.1	Turning a password into a key	6
4.2	Encrypting the entries	7
4.3	Storage: what actually touches disk	8
4.4	Master password verification	8
5	The Quantum Threat Model, In Detail	9
5.1	Shor’s algorithm vs. Grover’s algorithm	9
5.2	“Harvest now, decrypt later”	9
5.3	Why not apply post-quantum cryptography everywhere?	9
5.4	Why not Quantum Key Distribution (BB84)?	9
6	The Sync Protocol	10
6.1	Key agreement: the problem	10
6.2	X25519: the classical half	10
6.3	ML-KEM-768: the post-quantum half	10
6.4	Combining them: HKDF and the hybrid security argument	10
6.5	ML-DSA-65 and server authentication	11
6.6	Authorization: who may access a given vault	11
6.7	Transport security	12
6.8	Resource protection	12
6.9	Protocol walkthrough and replay protection	12
7	Supporting Engineering Practices	14
7.1	Crypto-agility	14
7.2	Module boundary enforcement	14
7.3	Memory hygiene	14
8	Verification and Validation	15
8.1	Static analysis	15
8.2	Adversarial and security testing	15
8.3	Benchmark: classical vs. hybrid handshake	16
8.4	Network trace verification	16
9	Limitations and Scope	17
9.1	Design trade-offs and residual technical limitations	17
9.2	Explicitly out of scope	17
10	Future Work	18
11	Conclusion	19
A	Appendix A: Reproduction	20

B Appendix B: Project Structure**20**

1 Executive Summary

This project is a desktop password manager split into two modules with a hard boundary between them: an **offline-first core vault** that has no network code path at all, and a **separate, opt-in sync module** that protects vault backups in transit using a hybrid classical/post-quantum handshake (X25519 + ML-KEM-768), per-vault authorization, and a fail-closed transport-security policy.

The central engineering discipline of this project was not “use post-quantum cryptography” but *deciding precisely where it belongs and where it does not*, and defending that boundary with a concrete threat model rather than a vague sense that quantum computers are a general threat to everything. Section 5 walks through that reasoning in detail, because it is the single most important cryptographic design decision in the project.

Cryptographic key agreement, however, only answers one question: how do two parties derive a shared secret. It says nothing about who is authorized to use that secret to read or write a given vault’s data, or whether the network path itself is protected independent of the handshake. The sync module treats these as three separate, layered concerns rather than assuming that a sound handshake implies a secure deployment: **key agreement** (Section 6), **authorization** (Section 6.6), and **transport security** (Section 6.7) are each designed and enforced independently.

What was built

- An offline vault: Argon2id key derivation with per-vault version tracking, AES-256-GCM entry encryption, SQLite storage with enforced owner-only file permissions, PyQt6 GUI, auto-lock, clipboard auto-clear, password generator.
- A hybrid sync protocol: X25519 + ML-KEM-768 key exchange with HKDF-SHA256, optional ML-DSA-65 server authentication with persistent trust-on-first-use key pinning, per-vault API-key authorization, and a transport-security layer that refuses to serve unencrypted traffic to non-local clients by default.
- Resource protection: request size limits, per-IP rate limiting on handshake initiation, and scheduled cleanup of expired session state.
- A test suite (pytest) covering core vault operations, sync protocol edge cases, and targeted adversarial scenarios (tampered ciphertext, malformed protocol messages, unauthorized access attempts, insecure-transport rejection).
- Static analysis (bandit, pip-audit), a benchmark comparing classical-only vs. hybrid handshake cost, and a packet capture confirming the protocol’s on-the-wire behavior matches its design.

2 Problem Statement

2.1 The problem a password manager exists to solve

Reusing passwords across services, or choosing memorable-but-weak passwords, is the largest practical cause of account compromise - not sophisticated cryptanalysis. A password manager's job is to make *one* strong secret (the master password) sufficient to protect many independent, high-entropy secrets, so the user never needs to remember or reuse the underlying credentials.

This creates a specific, concentrated risk: the vault file becomes a single high-value target. If an attacker obtains it, every credential it protects is only as safe as the cryptography guarding that one file. This motivates the two central questions the core vault had to answer:

1. How do we turn one human-memorable password into a key strong enough to protect that file, given that an attacker who steals the file can brute-force it completely offline, with no rate limiting, for as long as they want? (Section 4.1)
2. How do we encrypt the actual secrets so that even a single-bit tamper or a nonce mistake cannot silently corrupt or leak data? (Section 4.2)

2.2 The problem that motivated the sync module

A local-only vault is safe but not useful across devices. The moment a network is introduced, an entirely new class of problem appears:

- An attacker does not need to steal a file - they can passively observe or actively interfere with network traffic.
- A network attacker present *today* can record encrypted traffic and hold onto it, motivating the “harvest now, decrypt later” problem discussed in Section 5.
- A network-reachable server introduces questions a local file never had to answer: who is allowed to talk to it, and over what kind of connection.

Post-quantum cryptography addresses the first two points - it is a deliberate, scoped response to network key exchange being retroactively breakable by a future quantum computer, not a blanket “upgrade everything” choice. The third point is addressed separately, by the authorization and transport-security layers described in Sections 6.6 and 6.7: a sound key exchange is necessary for a secure sync module, but it is not sufficient on its own, and this project's architecture reflects that directly rather than treating it as an afterthought.

3 System Architecture

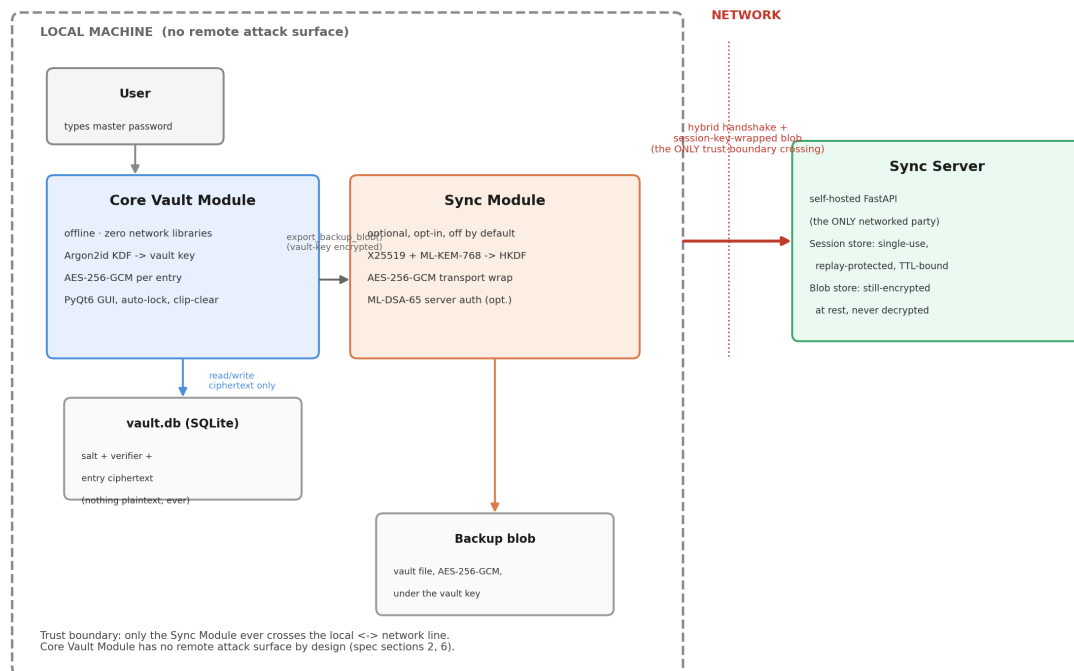


Figure 1: Two-module architecture. The core vault has no network code path; the sync module is the only component that ever crosses the local/network trust boundary, and only when explicitly enabled. Within the sync module, key agreement, authorization, and transport security are enforced as independent layers.

Design problem: how do you *prove* a module has no remote attack surface, rather than just asserting it in a comment?

The claim “the core vault cannot leak data over a network because it has no network code” is only credible if it is checked, not just documented. This project verifies it by direct import inspection: a test imports every core and GUI module (excluding the one on-click handler that opens sync settings) and asserts that no networking library (`httpx`, `fastapi`, `oqs`, `starlette`, `uvicorn`) is present in `sys.modules` afterward. This converts an architectural claim into an automated, falsifiable test - see Section 8.

4 Core Vault: Design and Limits

4.1 Turning a password into a key

The problem

A master password is a string a human can remember. An AES-256 key is 256 uniformly random bits. Naively hashing the password once (e.g. `SHA256(password)`) produces a key of the right *length* but not the right *security property*: SHA-256 is designed to be fast, and an attacker who has stolen the vault file can compute billions of SHA-256 hashes per second on commodity GPU hardware, making even a reasonably strong password crackable in a practical amount of time.

The technique: Argon2id

Argon2id is a *memory-hard* key derivation function: computing it requires allocating and repeatedly accessing a configurable amount of RAM (this project uses 64 MiB) for each single guess, not just CPU cycles. This defeats GPU/ASIC parallelization - an attacker can build a chip with thousands of parallel arithmetic units cheaply, but cannot cheaply give each of those units its own 64 MiB of fast memory. Argon2id specifically uses a hybrid access pattern that resists both GPU-style parallel attacks and certain cache-timing side-channel attacks, which is why it is the OWASP-recommended default over Argon2i or Argon2d.

Table 1: KDF alternatives considered

Candidate	Why considered	Why not chosen
PBKDF2	Widely supported, simple, FIPS-approved	Not memory-hard; purely CPU-bound iteration count, which is exactly what GPUs parallelize well.
bcrypt	Long track record, memory-hard-ish	Fixed, small memory cost (4 KB) by modern standards; cannot be tuned to today's attacker hardware the way Argon2 can.
scrypt	Also memory-hard, predates Argon2	Argon2 was the winner of the 2015 Password Hashing Competition specifically because it improved on scrypt's resistance to certain time-memory trade-off attacks.

Version-aware key derivation

Argon2id's parameters (memory cost, iteration count) are not fixed for the life of the project - hardware improves, and recommended cost parameters rise accordingly. Each vault records the parameter version that was in effect when it was created, and unlocking a vault always derives the key using *that* vault's own recorded version rather than whatever the software's current default happens to be. A dedicated migration path (`Vault.rekey_to_current_params()`) re-encrypts an existing vault's entries under a newer parameter set on request, re-encrypting entries and updating the stored version only after the new key has been verified to work, so an interruption mid-migration leaves the vault fully usable under its previous version rather

than in a half-migrated state. This is what makes it possible to strengthen the KDF's cost parameters over time without breaking previously-created vaults.

Limitations of this technique, stated plainly

- **Argon2id cannot protect a weak password.** If the user's master password is `password123`, no amount of memory-hard stretching makes brute force meaningfully harder - it only multiplies the cost per guess, not the size of the guess space. A `zxcvbn` strength check (minimum score 3/4) is enforced at vault creation as a separate, complementary defense.
- **Parameter tuning is a genuine trade-off.** 64 MiB / 3 iterations balances unlock latency against attacker cost; there is no setting that is simultaneously “instant for the user” and “infeasible for an attacker with unlimited offline time.”
- **No protection against a compromised endpoint.** If malware is running on the user's machine at the moment they type their master password, Argon2id's memory-hardness is irrelevant - the password is captured before it ever reaches the KDF (see Section 9.2).

4.2 Encrypting the entries

The problem

Confidentiality alone is not enough. If an attacker can flip bits in ciphertext and the application decrypts the result without noticing, the attacker gains a tampering primitive even without breaking confidentiality - and a naive combination of encryption and a separately-computed MAC can be implemented incorrectly, a class of bug that has caused real, exploitable vulnerabilities in production systems.

The technique: AES-256-GCM

AES-256-GCM is an AEAD (Authenticated Encryption with Associated Data) cipher: a single operation produces ciphertext *and* an authentication tag, and decryption fails closed - returning no plaintext at all - if the tag does not match, removing an entire class of implementation mistakes by construction.

Table 2: AEAD alternatives considered

Candidate	Why considered	Why not chosen
AES-256-CBC + HMAC	Well understood, used historically	Requires correct manual composition (encrypt-then-MAC); AEAD removes this footgun entirely.
ChaCha20-Poly1305	Faster in software on platforms without AES hardware acceleration	AES-NI is available on essentially all modern desktop/laptop CPUs this project targets, so AES-256-GCM was chosen as the more conventional, widely-audited default in this context.

Limitations of this technique, stated plainly

- **Nonce reuse under the same key is catastrophic for GCM specifically** - unlike some other modes, reusing a 96-bit nonce with the same key can allow full recovery of the authentication key and forgery of arbitrary ciphertexts. A fresh `os.urandom(12)` nonce is generated for every encryption operation, which is safe under the birthday bound for the volume of entries any single vault will realistically ever contain - at approximately 2^{32} encryptions under one key the collision risk becomes non-negligible, requiring billions of entries in a single vault.
- **No post-compromise security.** If the vault key is ever extracted from memory, every past and future ciphertext encrypted under that key is retroactively readable. AES-256-GCM provides no forward secrecy on its own - that property is engineered into the *sync* protocol instead (Section 6) and does not exist for local storage.
- **Grover's algorithm margin is an estimate, not a certainty.** AES-256's safety against a quantum adversary rests on Grover's algorithm providing only a quadratic speedup, giving an effective ~ 128 -bit security level - the mainstream cryptographic consensus, but a projection about hardware that does not yet exist at scale.

4.3 Storage: what actually touches disk

The SQLite file stores exactly three kinds of data: the Argon2id salt (not secret - its purpose is to be public and unique per vault), an encrypted verifier canary (below), and, per entry, a nonce and an AES-GCM ciphertext blob. Plaintext password data and the derived vault key are never written to disk or logs.

The storage layer enforces owner-only (0600) file permissions on the vault file, both when it is first created and every time it is subsequently opened, so a vault created under a permissive umask - or copied in from elsewhere - is tightened automatically rather than silently left group- or world-readable.

Limitation: metadata leakage

Even though entry *contents* are fully encrypted, the SQLite file's structure leaks metadata: entry count, approximate entry sizes (ciphertext length tracks plaintext length closely for GCM), and creation/modification timestamps are visible to anyone who obtains the file, without the master password. For most personal use this is a minor concern; closing it fully would require changing the storage format itself (padding, oblivious access patterns), which was judged out of scope for the value it would add here.

4.4 Master password verification

Unlocking a vault verifies the candidate password against a small, fixed plaintext constant (a "verifier canary") that is encrypted under the vault key at creation time and stored independently of whether any real entries exist. This is deliberate: verifying against the vault's first real entry would leave a brand-new, empty vault with nothing to check a candidate password against, silently accepting any password at unlock time until the first entry read revealed the mistake. The verifier canary gives unlock a definite answer regardless of how many entries the vault contains.

5 The Quantum Threat Model, In Detail

This section exists because “post-quantum cryptography” is frequently invoked as a feature without a precise explanation of what it defends against, at what cost, and where it does and does not apply. This project treats that precision as the actual point of the exercise.

5.1 Shor’s algorithm vs. Grover’s algorithm

- **Shor’s algorithm** efficiently solves integer factorization and the discrete logarithm problem on a sufficiently large, fault-tolerant quantum computer. Every widely-deployed classical public-key primitive - RSA, Diffie-Hellman (including elliptic-curve variants such as X25519) - derives its security from exactly these two hard problems, which Shor’s algorithm breaks outright rather than merely weakens.
- **Grover’s algorithm** provides a generic quadratic speedup for unstructured search, including brute-forcing a symmetric key: an n -bit key’s effective security drops to roughly $n/2$ bits under a quantum attacker. For AES-256, that is a drop to ~ 128 -bit security - still comfortably infeasible to brute-force by any foreseeable computer.

This distinction is the entire justification for the project’s scoping decision: everywhere this project uses only symmetric cryptography (Argon2id output feeding AES-256-GCM), Grover’s algorithm is the relevant quantum threat, and AES-256’s margin already absorbs it. Post-quantum replacement algorithms exist specifically to replace what Shor’s algorithm breaks - key exchange and digital signatures - which is why ML-KEM-768 and ML-DSA-65 appear *only* in the sync handshake and nowhere in local storage.

5.2 “Harvest now, decrypt later”

An adversary who can record network traffic today gains value from doing so even before a cryptographically relevant quantum computer exists: ciphertext protected only by classical key exchange can be stored indefinitely and decrypted retroactively once such a computer becomes available. This present-day risk with a future payoff is why the sync handshake - the one place this project’s traffic crosses an untrusted network - uses post-quantum key exchange. Local vault storage has no equivalent exposure: there is no ciphertext in transit for a network observer to harvest.

5.3 Why not apply post-quantum cryptography everywhere?

Applying PQC to local storage as well would add real complexity - larger keys, slower operations, more code to audit - against a threat (Grover’s algorithm against AES-256) that is not considered practically exploitable. Security engineering is a cost/benefit exercise: applying PQC where it provides no meaningful benefit is added attack surface for no corresponding gain.

5.4 Why not Quantum Key Distribution (BB84)?

QKD protocols such as BB84 provide information-theoretic security for key exchange, a stronger guarantee than any software-only PQC scheme. It was rejected for a practical reason: BB84 requires specialized photonic hardware and a dedicated physical channel between the communicating parties, making it infeasible for a self-hosted sync server reached over an ordinary network connection.

6 The Sync Protocol

The sync protocol is composed of three independent layers: key agreement (this section), authorization (Section 6.6), and transport security (Section 6.7). Each answers a different question, and each is enforced independently rather than assumed to follow from the others.

6.1 Key agreement: the problem

Two parties - a user's vault and their self-hosted sync server - need to agree on a shared secret over a network neither fully trusts, without having exchanged any secret material in advance, and with that agreement remaining secure even against an adversary who can record today's traffic and attack it with tomorrow's computer.

6.2 X25519: the classical half

X25519 is Curve25519-based Diffie-Hellman key exchange: mature (in production use since the mid-2010s across TLS, SSH, Signal, WireGuard), fast, small (32-byte keys), and extremely well-analyzed.

Limitation

X25519 alone provides *zero* protection against “harvest now, decrypt later” - it is exactly the primitive Shor's algorithm breaks completely. It remains in this design specifically *because* of its maturity: the well-understood, heavily-scrutinized half of the hybrid combination.

6.3 ML-KEM-768: the post-quantum half

ML-KEM (the NIST-standardized version of the algorithm formerly known as Kyber) is a lattice-based Key Encapsulation Mechanism, whose presumed hardness rests on the Module Learning With Errors (M-LWE) problem.

Limitations, stated plainly

- **Much less battle-tested than X25519.** ML-KEM was only finalized as a NIST standard in 2024, which is precisely why the design does not trust it alone - see Section 6.4.
- **Larger keys and ciphertexts.** A public key is 1184 bytes and a ciphertext is 1088 bytes, versus 32 bytes each for X25519.
- **Non-zero decryption failure probability**, on the order of 2^{-164} or smaller - negligible in practice, but a qualitatively different failure mode than the classical half.
- **Relies on the Fujisaki–Okamoto transform** for its strong (IND-CCA2) security guarantee, an additional layer of construction beyond the base lattice problem.

6.4 Combining them: HKDF and the hybrid security argument

The X25519 shared secret and the ML-KEM-768 shared secret are concatenated and passed through HKDF-SHA256 to derive the final AES-256-GCM session key. This construction's security argument is simple and important: the combined key is secure as long as at least one of the two input secrets is secure. Neither algorithm is a single point of failure for the other. The HKDF context (`info` parameter) is derived from an explicit protocol version number together with the per-handshake session identifier, so different protocol versions and different sessions can never derive colliding key material from the same input secrets.

Limitation

This construction adds implementation complexity compared to a single-algorithm handshake and makes every handshake message larger (Section 8.3 measures this at roughly 2.3 KB of key material per handshake versus 64 bytes classical-only). For this project's actual workload - infrequent, user-initiated backup sync - that overhead is negligible.

6.5 ML-DSA-65 and server authentication

Key exchange alone does not tell the client *who* it just negotiated a shared secret with. ML-DSA (the NIST-standardized version of Dilithium) is a lattice-based signature scheme used to let the server sign its ephemeral handshake material, which the client verifies before trusting the exchanged keys.

Because this project has no certificate authority infrastructure, the client uses **trust-on-first-use (TOFU) key pinning**: it remembers the server's signing public key on first connection and aborts if a future connection presents a different one, the same trust model SSH uses for host keys. The server persists its signing keypair to disk with owner-only permissions, so a returning client's pinned key remains valid across server restarts.

Whether server authentication is required is determined entirely by the *client's own configuration*, not by whether a given handshake response happens to include a signature. A client configured to require server authentication - the default - will reject any handshake response that lacks one, rather than inferring the requirement from the response's contents. This matters because a response's contents are, by definition, something an active network attacker can manipulate in transit; a security-critical decision should never be delegated to data that an adversary controls.

Limitation

TOFU is, by construction, vulnerable to an attacker already positioned as a man-in-the-middle during the very first connection - there is nothing to detect a substitution if no prior key was ever pinned. This is a known, accepted trade-off of TOFU generally (SSH has the same limitation), not a flaw specific to this implementation.

6.6 Authorization: who may access a given vault

Key agreement establishes that two parties share a session key; it does not establish that the calling client is *allowed* to push or pull a specific vault's backup blob. The sync server therefore maintains a separate, explicit access-control layer: each `vault_id` must be registered on the server with a high-entropy API key before any client can sync against it. Registration is an out-of-band operator action - running `python -m sync.manage_access register <vault_id>` generates and prints a key, which is then configured into the client - the same operational shape as adding an entry to an SSH `authorized_keys` file.

This is a deliberately different trust model from TOFU key pinning: pinning defends against a passive attacker on the first connection by trusting whatever key is presented *once*, which is an acceptable trade-off when the alternative is no continuity check at all. Auto-registering *authorization* on first contact would not be an acceptable analogue, since it would let any client that reaches the server first simply claim an arbitrary `vault_id` for itself. Authorization is therefore always an explicit registration, never inferred from first contact.

The server stores only a salted hash of each API key, using a constant-time comparison for verification, and checks authorization before any session state or ephemeral key material is

generated for a handshake - an unregistered or incorrectly-keyed client cannot cause the server to do cryptographic work on its behalf, not merely be blocked at the point of actually reading or writing data.

Limitation

Key rotation and revocation are manual, via the same CLI, with no expiry policy or audit log. This is reasonable scope for a self-hosted, single-operator tool; a multi-user deployment would need more infrastructure around key lifecycle management (Section 10).

6.7 Transport security

A sound handshake and a correct authorization check both assume the underlying network connection is not itself the attacker's vantage point. The sync server enforces this assumption directly rather than leaving it to deployment convention: any request that is neither from a loopback client address nor verifiably encrypted - either because TLS terminates at this process directly, or because a reverse proxy in front of it sets a trusted `X-Forwarded-Proto: https` header - is rejected outright rather than served in plaintext.

The server can terminate TLS itself, given a certificate and key (`PQVAULT_SYNC_TLS_CERT` / `PQVAULT_SYNC_TLS_KEY`), for the simplest single-binary self-hosted deployment, or it can sit behind a reverse proxy that handles TLS termination and forwards the appropriate header. Binding to a non-local address without either TLS configured or an explicit operator override is refused at startup.

Limitation

This project cannot provision a valid TLS certificate on the operator's behalf - obtaining one (e.g. via a public CA) or standing up a trusted reverse proxy remains an operational responsibility. Trusting a proxy's `X-Forwarded-Proto` header is only sound if this server is never directly reachable, bypassing the proxy - the standard caveat that applies to any use of `X-Forwarded-*` headers, not something specific to this implementation.

6.8 Resource protection

The server enforces a hard cap on request body size, checked against bytes actually received rather than a client-supplied `Content-Length` header, so an oversized or maliciously-framed request cannot be used to exhaust server memory. Handshake initiation is separately rate-limited per source address, since that endpoint is the one that creates server-side session state and performs cryptographic key generation on each call; the same limiter also bounds the rate at which an incorrect authorization key can be attempted.

6.9 Protocol walkthrough and replay protection

1. Client requests a handshake, authenticated with its per-vault API key (Section 6.6); server verifies authorization, then generates *ephemeral* X25519 and ML-KEM-768 keypairs, a random 128-bit `session_id`, and signs the transcript.
2. Client verifies the signature, generates its own ephemeral X25519 keypair, encapsulates against the server's ML-KEM-768 public key, and derives the session key.
3. Client sends its public key and the KEM ciphertext back to the server, which derives the identical session key independently.

4. The session transitions from **PENDING** to **ESTABLISHED**. A single upload or download then consumes it, transitioning to **CONSUMED** - any further use of that `session_id` is rejected.

This single-use session ID is the primary replay defense. Secondary defenses - handshake and transfer expiry windows, and a clock-skew check - bound how long a session can be left dangling; a background task periodically sweeps expired sessions and stale rate-limiter entries so abandoned handshakes do not accumulate indefinitely between lookups.

Limitation

The session store, rate limiter, and authorization store are in-memory or single-file, tied to a single running server process. A restart drops in-flight sessions safely (clients retry with a fresh handshake), but none of this extends to a multi-instance or load-balanced deployment without a shared backing store.

7 Supporting Engineering Practices

7.1 Crypto-agility

Every algorithm choice and cost parameter lives in a single configuration module (`core/crypto.config.py`); no cost parameter, key size, or algorithm name is hardcoded elsewhere. Argon2id parameters are additionally versioned per Section 4.1, so a future increase in cost parameters does not require every existing vault to be re-created - only, optionally, migrated.

7.2 Module boundary enforcement

The core/sync boundary is checked by an automated test, not merely documented (Section 8) - the project's central safety claim, that the core vault cannot leak data over a network, is only as strong as this boundary being real.

7.3 Memory hygiene

The vault key and clipboard contents are overwritten with zero bytes as soon as they are no longer needed.

Limitation

This is explicitly *best-effort*, not a guarantee. CPython does not provide a way to guarantee that sensitive data was never copied elsewhere in memory - string immutability, the garbage collector, and OS-level swapping to disk can all leave copies this project's zeroing code has no visibility into. A systems language with explicit memory control, or a hardware secure enclave, would be required for a hard guarantee.

8 Verification and Validation

8.1 Static analysis

bandit found zero Medium/High severity issues across the codebase. Eight Low-severity findings were individually reviewed rather than blanket-suppressed:

Table 3: Static analysis findings and disposition

Finding	Location	Disposition
B110 (try/except/-pass)	core/clipboard.py	Deliberate best-effort exception swallow inside a background timer callback with no caller to report a failure to.
B105 (hardcoded password) $\times 2$	core/kdf.py, gui/entry_dialog.py	False positive - both are empty-string equality checks, not credentials.
B101 (assert used) $\times 5$	sync/pqc_test_kem.py, sync/pqc_test_sig.py	Standalone verification scripts, never imported by the application itself.

pip-audit identified known CVEs in several pinned third-party packages, including **cryptography** and a transitively-pulled-in **starlette** version. These were remediated by upgrading the affected packages to patched releases; the full regression suite was re-run afterward and passed, confirming the relevant API surface (AESGCM, X25519, HKDF, FastAPI routing) remained stable across the upgrade.

8.2 Adversarial and security testing

The test suite covers core vault operations and sync protocol behavior under both normal and adversarial conditions: empty and oversized inputs, Unicode and SQL-injection-shaped strings, directly tampered ciphertext and nonces, and malformed protocol messages. All handshake and sync endpoints validate and catch cryptographic-library exceptions at the request boundary, returning a clean HTTP 400 for malformed input rather than allowing an internal exception to propagate and potentially surface implementation details.

A dedicated set of security-focused tests exercises the layered design described in Sections 6.6–6.7 directly: a handshake response lacking a server signature is rejected by a default-configured client; an unregistered **vault_id** or incorrect API key is rejected by the server before any session state is created; a plaintext request from a non-local address is rejected by the transport-security layer, while the same request is accepted when carrying a trusted **X-Forwarded-Proto: https** header; and a newly created or reopened vault file is confirmed to carry owner-only permissions.

8.3 Benchmark: classical vs. hybrid handshake

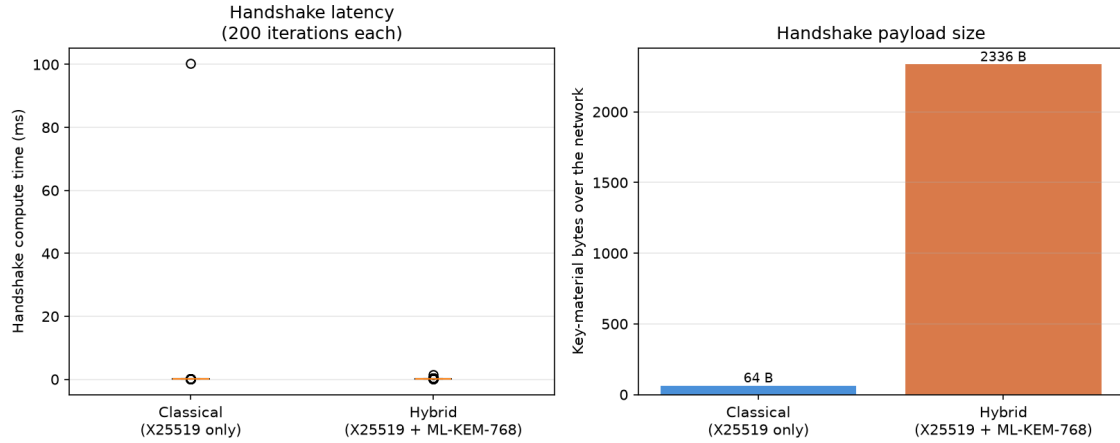


Figure 2: 200 iterations each. Left: latency distribution (box plot, outliers shown). Right: key-material payload size.

Table 4: Benchmark results

Metric	Classical (X25519)	Hybrid	Overhead
Median latency	0.304 ms	0.412 ms	+0.108 ms
Mean latency	0.384 ms	0.459 ms	+0.075 ms
Payload (key material)	64 bytes	2,336 bytes	+2,272 bytes

The latency overhead is negligible in absolute terms - both variants complete in a fraction of a millisecond, dwarfed by any real network round-trip time. The payload overhead is real but small for a workload that runs, at most, a handful of times per day. These figures reflect handshake computation only; the request-size and rate-limiting mechanisms in Section 6 operate on request framing rather than the handshake payload itself, so the numbers above are unaffected by them.

8.4 Network trace verification

A `tcpdump` capture was taken on the loopback interface while a real `uvicorn` server and a real `SyncClient` performed a full upload and download round trip. 54 packets were captured; parsing the JSON bodies confirmed a genuine 1184-byte ML-KEM-768 public key and 1088-byte ciphertext on the wire, and that the upload and download used two independent handshakes with distinct key material - confirming the protocol's on-the-wire behavior matches its design, and that session keys are never reused across transfers.

9 Limitations and Scope

This section consolidates, in one place, what this project deliberately does not solve. Individual limitations are also noted alongside the relevant design discussion throughout the report; this section exists so the overall scope is visible without reading every section in full.

9.1 Design trade-offs and residual technical limitations

- Argon2id cannot compensate for a weak master password (Section 4.1).
- Local storage metadata (entry count, timestamps) is not hidden by entry-level encryption (Section 4.1).
- Memory zeroing is best-effort, not a hard guarantee (Section 4.1, “Memory hygiene”).
- AES-256’s post-quantum security margin is a projection, not a proof (Section 4.2).
- TOFU key pinning cannot detect an attacker already positioned as a man-in-the-middle before the first connection (Section 6).
- The session store, rate limiter, and authorization store are single-process and in-memory or single-file; a multi-instance deployment needs a shared backing store (Section 6).
- TLS certificate provisioning is an operational responsibility this project supports but cannot itself fulfill (Section 6.7).
- API key rotation and revocation are manual, with no expiry policy or audit log (Section 6.6).

9.2 Explicitly out of scope

- A compromised operating system or a keylogger on the user’s machine, which can capture the master password regardless of any cryptography downstream of that capture.
- Physical device theft while the vault is unlocked.
- Supply-chain attacks against dependencies, beyond version pinning and `pip-audit`.
- Side-channel attacks against the underlying `cryptography` or `liboqs` library implementations themselves - this project relies on those libraries’ own hardening rather than adding independent countermeasures.
- The vault file being exposed via a third-party sync tool the user runs outside this application (Dropbox, iCloud, a shared network drive, and similar). File permissions on disk are enforced by this project (Section 4.1), but detecting or warning about a user’s own choice to place the vault file inside a folder managed by unrelated software is inherently environment-specific and outside what this project addresses.
- Master password recovery or multi-factor unlock. Every guarantee in this system ultimately rests on the secrecy and strength of one string, by design - a recoverable master password would itself be a security weakness, though it is a real usability/availability trade-off worth stating plainly.
- Formal, machine-checked verification of the sync protocol (e.g. via Tamarin or ProVerif). The protocol was designed against a written threat model and validated with targeted adversarial testing, which is appropriate rigor for this project’s scope but is not equivalent to exhaustive automated state-space analysis.

10 Future Work

1. Key rotation, expiry, and audit logging for per-vault authorization (Section 6.6).
2. Mutual TLS between a reverse proxy and the sync server, as a stricter alternative to trusting a forwarded-protocol header (Section 6.7).
3. A shared backing store (e.g. Redis) for the session store, rate limiter, and authorization store, to support multi-instance deployments.
4. Formal verification of the handshake protocol using a tool such as Tamarin or ProVerif.
5. Investigation of optional multi-factor unlock for the master password.
6. Replacing the custom handshake with TLS 1.3 hybrid PQC via OpenSSL 3.x and `oqs-provider`, which would inherit a much larger body of real-world scrutiny than a bespoke protocol.

11 Conclusion

The strongest claim this project can make is not “it uses post-quantum cryptography” but that every cryptographic and architectural decision in it can be traced back to a specific problem, a specific alternative that was rejected and why, and a specific limitation that remains even after the chosen solution was adopted. Section 5’s scoping of post-quantum cryptography to exactly the place it is needed - and nowhere else - is the clearest example of that discipline, but the same reasoning shapes the sync module’s layered design: key agreement, authorization, and transport security are treated as three separate problems with three separate answers, because a sound handshake does not, by itself, make a deployed system secure. Section 9 states plainly what is left - items that are either deliberate design trade-offs or genuinely outside what application code alone can resolve, such as certificate provisioning or distributed infrastructure - so that the system’s actual guarantees, and the boundaries of those guarantees, are both clear.

A Appendix A: Reproduction

```
# Build toolchain
sudo apt-get install -y cmake ninja-build build-essential libssl-dev
pip install -r requirements.txt

# Build liboqs (minimal: ML-KEM-768 + ML-DSA-65 only)
git clone --depth 1 https://github.com/open-quantum-safe/liboqs.git
cd liboqs && mkdir build && cd build
cmake -GNinja -DCMAKE_INSTALL_PREFIX=$HOME/_oqs \
  -DBUILD_SHARED_LIBS=ON \
  -DOQS_MINIMAL_BUILD="KEM_ml_kem_768;SIG_ml_dsa_65" \
  -DOQS_BUILD_ONLY_LIB=ON ..
ninja && ninja install
export LD_LIBRARY_PATH=$HOME/_oqs/lib:$LD_LIBRARY_PATH

# Register a vault for sync access (once per vault, before first sync)
python -m sync.manage_access register <vault_id>

# Run tests, static analysis, benchmark
python3 -m pytest tests/ -v
bandit -r core gui sync cli.py
pip-audit -r requirements.txt
python3 benchmarks/handshake_benchmark.py

# Run the sync server (enforces the transport-security policy; set
# PQVAULT_SYNC_TLS_CERT / PQVAULT_SYNC_TLS_KEY to terminate TLS directly,
# or place a reverse proxy in front and set
# PQVAULT_ALLOW_INSECURE_TRANSPORT=1 to trust its X-Forwarded-Proto header)
python -m sync.server
```

Full instructions, including the liboqs build rationale, are in docs/pqc_setup.md in the project repository.

B Appendix B: Project Structure

```
core/ vault, crypto, storage -- zero network dependencies
gui/ PyQt6 application
sync/ hybrid handshake, FastAPI server, sync client,
      access_store.py (per-vault authorization),
      manage_access.py (authorization CLI)
tests/ pytest suite: core + sync edge cases, security/regression
      coverage for authorization and transport-security behavior
benchmarks/ classical-vs-hybrid handshake benchmark + results
docs/ threat models, protocol design, PQC setup, trace capture
```